OFAC Screening Enterprise - Architecture and API Flow

Updated for the current implementation on 2026-03-23.

1) Deployed Architecture

AWS Account / Environment User Browser React SPA CloudFront HTTPS entry point ALB Frontend target group Frontend Service ECS/Fargate Nginx + SPA + localStorage state Backend Service FastAPI file-only batch parsing auth + audit + access logs Worker Service ECS/Fargate SQS consumer + scheduler DB pool + retry/backoff Cognito / IdP OIDC auth code + PKCE Cloud Map backend.screening.internal S3 Upload Storage source file backend-generated queries.json SQS JOB_DISPATCH SCREEN_ITEM messages RDS PostgreSQL jobs / items / schedules audit / access / API errors business-unit mappings CloudWatch Logs API access logs raw Actimize request/response worker structured failures Prudential / Actimize HTTP 200 + PM/NM contract single-type screening API

Browser traffic enters through CloudFront and ALB, reaches the frontend task, and `/api/*` is reverse-proxied to FastAPI through Cloud Map. Immediate large batch uploads are now file-only from the browser; the backend parses the file, stores the source plus generated `queries.json`, and dispatches async work through SQS.

2) API Flow Patterns

Single Screening (Sync) Frontend FastAPI Prudential / Actimize RDS + CloudWatch POST /api/v1/screenings/match single-type screening request audit + access log + raw request logging HTTP 200 + PM/NM, else fail Immediate UI result Batch Upload (Async, File-Only Upload Contract) Frontend FastAPI S3 SQS Worker RDS / Actimize POST /batch-upload (file + metadata) store source + generated queries.json create batch_file_uploads + jobs + metadata enqueue JOB_DISPATCH 202 Accepted + row_meta worker receives JOB_DISPATCH download generated queries.json bulk insert job_items enqueue SCREEN_ITEM batches screen + persist per-item result GET /jobs/{job_id} poll

The browser no longer sends `queries_json`. FastAPI parses the uploaded file, validates fields such as `PartyKey`, gender, and names, generates normalized screening payloads, and only then dispatches async work. Under load, the client can still time out at the gateway while the backend completes the job creation path, so job existence must be confirmed server-side for support cases.

3) Database Design (Core Tables)

jobs job_id, status, total_items source_schedule_id, source_upload_id user_id, user_name job_items PK: job_id + item_key request_json, response_json status, error_text, updated_at per-record screening execution job_metadata mode, screening_types_json batch_name, file_name, daily_schedule_id query_count, deferred_until business_unit_code batch_file_uploads upload_id, file_name, file_hash, record_count s3_uri, queries_s3_uri, user context schedule_id, job_id, is_active source-file traceability daily_schedules schedule_id, batch_name queries_json, screening_types_json timezone, run_hour, run_minute last_run_at, next_run_at, is_active source_upload_id, business_unit_code schedule_subscriptions subscription_id, schedule_id email, is_active created_at, updated_at schedule_record_state schedule_id + record_hash first_seen_at, last_screened_at last_job_id audit / ops tables audit_events api_access_logs external_api_errors schedule_notifications business_units business_unit_code business_unit_name, is_active created_at, updated_at user_business_units user_id + business_unit_code user_name, is_active created_at, updated_at

The execution core is `jobs` plus `job_items`, with `job_metadata` carrying UI/history fields. Scheduled runs, uploads, business-unit authorization, and audit/ops telemetry are stored in dedicated tables so each concern can be queried and retained independently.